

Karigar – Project Documentation

AI-Driven Market Linkage & Smart Cataloging for Marginalized Artisans – SIH 2026 / PS 26090
– Ministry of Social Justice & Empowerment

Prototype build – generated 2026-09-03

Contents

1. What Karigar is	2
2. What is working today	2
3. Technology stack – what is used, and where	3
3.1 Layers	3
3.2 Per-feature: which technique, where	3
4. Where AI is used – and how honest it is	6
5. What external APIs / keys you need	7
5.1 To run the prototype: NONE	7
5.2 Optional integrations (each purely additive)	7
5.3 Mobile-side “API” it must reach	8
6. Backend API reference	8
6.1 Core (F1–F7)	8
6.2 Trust & Verification (F8, new)	9
7. User flows	10
7.1 Artisan – create a listing (the main F1–F7 demo)	10
7.2 Buyer – bulk order via cluster pooling (the F6 demo)	10
7.3 Trust & Verification (the second, trust-focused demo)	10
7.4 Language	11
8. Known issues – what errors, why, and the fix	11
8.1 “No connection to the server” when you tap Login / any API screen	11
8.2 The camera shows a moving 3D room, not a real photo	11
8.3 Voice note always produces the same transcript	11
8.4 “Pay advance” / “Order protection & payments” – is this real money?	11
8.5 The AI photo enhancer can make a good photo worse	11
8.6 A buyer cannot place a small direct order (1–10 pieces)	11
8.7 Delivery / logistics tracking is partial	12
8.8 Cosmetic items already fixed this iteration	12
9. Setup and run	12
9.1 Prerequisites	12
9.2 Backend	12
9.3 Mobile	12
9.4 Windows-specific: Android build fails with “different roots”	12
9.5 Tests	13

10. What changed in this iteration	13
10.1 New – Trust & Verification (F8)	13
10.2 New – 22-language localization	13
10.3 UX + build fixes this iteration	13
11. Production upgrade path (brief)	14

1. What Karigar is

Karigar is a demo-ready mobile prototype that turns **one phone photo** and **one voice note** into a trustworthy, fairly-priced, fairly-discoverable, bulk-order-ready product listing – an “AI business manager” for artisans with low digital literacy, limited English and a low-end Android phone.

The build implements the seven original features (F1–F7) of the project’s Master Technical Feature Specification, plus a new **F8 – Trust & Verification** system and **22-language UI localization** added in this iteration.

Architecture (one line): Flutter mobile app -> REST/JSON -> FastAPI backend -> 7 swappable AI services + SQLAlchemy (SQLite by default) + OR-Tools + LightGBM.

Every AI call returns a technical-metadata object (**feature, model, mode, latency_ms, inputs**) that the app’s “AI System Insights” screen renders for the technical judging round.

2. What is working today

Legend for AI mode: **REAL** = runs live on real inputs; **PROTOTYPE** = deliberate heuristic stand-in behind the same interface; **SIMULATED_DATA** = real model, generated history; **FALLBACK** = deterministic path when an optional dependency is absent.

#	Feature	Status	AI mode
F1	AI Product Studio + Photo Readiness Gate	Working	REAL (OpenCV)
F2	Multilingual Grounded Auto-Cataloger	Working	REAL grounding / PROTOTYPE cross-modal / FALLBACK ASR
F3	Craft Knowledge Graph + Digital Passport	Working	REAL; certificates SIMULATED_DATA (labelled)
F4	Fair Cost-Aware Pricing	Working	REAL (LightGBM + tree-SHAP); FALLBACK formula if model file absent
F5	Demand & Opportunity Intelligence	Working	REAL model / SIMULATED_DATA history
F6	B2B Matching + Cluster Order Pooling	Working	REAL (OR-Tools CP-SAT MILP + exact Shapley)
F7	Fair Market Discovery + Business Copilot	Working	REAL (TF-IDF + fairness re-rank)
F8	Trust & Verification (Verified Artisan / Business)	Working	REAL state machine + reliability + risk-flag; PROTOTYPE simulated payments
–	22-language UI switcher (12 fully translated)	Working	n/a

Test status of this build

- Backend: `pytest` – 33 passed. `scripts/smoke.py` – 43/43 checks passed.
- Mobile: `flutter analyze` – 0 issues. `flutter test` – 5/5 passed. `flutter build web` – OK.
- Full API smoke of every endpoint incl. the payment/commitment path – 0 failures.
- Android APK builds and installs on the emulator (after the Windows fix in section 9.4).

3. Technology stack – what is used, and where

3.1 Layers

Layer	Technology	Where in the repo
Mobile app	Flutter (Dart 3.13), provider (state), go_router (navigation), http (networking), image_picker , record 6.x (mic), audioplayers , fl_chart , shared_preferences , flutter_localizations + intl	<code>mobile/lib/</code>
Backend / BFF	Python 3.12–3.14 + FastAPI + SQLAlchemy 2 + Pydantic v2 + Uvicorn	<code>backend/app/</code>
Database	SQLite (zero-config default file <code>backend/data/sih.db</code>) or PostgreSQL + pgvector via <code>docker-compose</code>	<code>backend/app/db/</code> , <code>backend/app/models/</code> (33 tables)
Computer vision	OpenCV (headless): GrabCut segmentation, Laplacian variance, histogram analysis, CLAHE, Canny	<code>backend/app/ai/f1_studio/analyzer.py</code>
Machine learning	LightGBM (regression + quantile regression), scikit-learn (TF-IDF), NumPy	<code>backend/app/ai/f4_pricing/</code> , <code>f5_demand/</code> , <code>f7_ranking/</code>
Optimization	Google OR-Tools CP-SAT (MILP); exact Shapley value by 2^N enumeration	<code>backend/app/ai/f6_matching/allocation.py</code>
Speech (optional)	pluggable: faster-whisper OR an OpenAI-compatible <code>/audio/transcriptions</code> API; otherwise a deterministic demo transcript	<code>backend/app/ai/f2_catalog/asr.py</code>
Text generation	Deterministic EN/Hi templates; an optional OpenAI-compatible LLM only <i>rephrases</i> already-validated facts	<code>backend/app/ai/f2_catalog/cataloger.py</code>
Auth	Mock phone-OTP -> JWT (PyJWT), HS256	<code>backend/app/api/routes/auth.py</code> , <code>backend/app/core/security.py</code>
Localization	In-app dictionary lookup <code>tr("English source")</code> , LocaleProvider persisted via SharedPreferences	<code>mobile/lib/l10n/i18n.dart</code> , <code>mobile/lib/l10n/langs.dart</code>

3.2 Per-feature: which technique, where

Feature	Core method	File
F1	EXIF-orient decode; GrabCut foreground mask; 5 measured dimensions – variance-of-Laplacian (focus), luminance histogram mean + tail-clip (exposure), mask bbox ratio + centroid (framing), LAB colour-std + Canny edge density in the mask complement (background), pixel count (resolution); weighted 0–100 composite; 3-way gate accept ≥ 80 / auto-enhance 50–79 / retake < 50 . Auto-enhance = measured-gamma tone lift + CLAHE on L + grey-world white balance + masked studio-gradient background, then re-score.	backend/app/ai/f1_studio/analyzer.py
F2	ASR (pluggable) -> rule-based slot NER seeded with the Craft-KG vocabulary -> Fact-Grounding Guard (an attribute is <code>confirmed_from_voice</code> only if its token is in the transcript; any origin/GI claim is always <code>flagged</code>) -> colour/category cross-modal heuristic -> EN/HI template copy + SEO keywords; optional LLM rephrase of validated slots only.	backend/app/ai/f2_catalog/asr.py, cataloger.py
F3	Relational craft graph (region / cluster / craft / technique / material / passport / certification); “related crafts” via a genuine recursive CTE over a typed <code>craft_relation</code> edge table (runs identically on SQLite and Postgres); <code>provenance_confidence = 0.40*gov + 0.25*grounding + 0.20*visual + 0.15*self_report</code> , persisted with its component breakdown.	backend/app/ai/f3_graph/graph.py
F4	Cost floor <code>P_floor = base_cost / (1 - m_min)</code> ; LightGBM regressor predicts a market price; <code>recommendation = max(P_floor, clip(model, P_floor, P_market_max))</code> – structurally impossible to recommend below the floor (400 randomised unit tests confirm); native tree-SHAP for the per-prediction “why”.	backend/app/ai/f4_pricing/pricing.py, features.py, train.py

Feature	Core method	File
F5	Three LightGBM quantile models (alpha = 0.1 / 0.5 / 0.9) rolled forward over the window; point forecast = 0.5 sum, interval = 0.1/0.9 sums; a decision layer converts the forecast delta into “make N more units by DATE”, capped at the artisan’s monthly capacity. History is generated (~200 days/category) and the UI says so.	backend/app/ai/f5_demand/forecast.py, train.py
F6	Eligibility filter (craft compatibility via F3, capacity, price band, verification status , reliability) BEFORE the optimizer; CP-SAT model minimises $\text{sum}(\text{cost} + \text{logistics}) * x_i - \lambda * \text{sum}(\text{quality} + \text{reliability}) * x_i$ s.t. demand, per-artisan capacity, min-lot, max-artisans; switches to maximise-fill and reports the shortfall if capacity < demand. Buyer payment then split by the exact Shapley value over all 2^N sub-coalitions (tractable at $N \leq 8$), shown against a by-units split and a greedy baseline.	backend/app/ai/f6_matching/allocation.py, backend/app/services/helpers.py
F7	TF-IDF (1-2 gram) cosine + keyword overlap retrieval; re-rank $\alpha * \text{relevance} + 0.15 * \text{quality} + \beta * \text{newSellerBoost} + \gamma * \text{regionBoost} - \delta * \text{exposurePenalty}$; newSellerBoost decays linearly to 0 as verified-sales approach a threshold; one impression persisted per shown listing so exposure state evolves; position-weighted exposure-gap metric + fairness ON/OFF comparison. Business Copilot cards are built from structured F1/F4/F5/F6 output (LLM only phrases).	backend/app/ai/f7_ranking/ranking.py, backend/app/api/routes/discovery.py

Feature	Core method	File
F8	Per-axis verification (identity / craft / product-claims / business / contact) each backed by evidence rows; GI status on its own axis (not_registered / artisan_reported / pending_verification / government_verified) – never inferred from craft or region; trust-status state machine PENDING -> VERIFIED -> UNDER_REVIEW -> SUSPENDED -> REINSTATED (+ REJECTED) with enforced transitions; explainable reliability score (0–100, every term sums to the total) from real order + complaint signals; complaint -> advisory AI risk flag -> UNDER_REVIEW -> human decision (AI never bans); simulated B2B order-commitment + milestone payments (20/30/30/20) + stage-aware cancellation.	backend/app/models/trust.py, backend/app/services/trust.py, backend/app/api/routes/trust.py

4. Where AI is used – and how honest it is

The codebase labels every AI component and surfaces it on the “AI System Insights” screen.

Component	Mode	Note
F1 segmentation + 5-dim scoring + gate	REAL	OpenCV on the actual pixels. GrabCut stands in for SAM (documented production upgrade).
F1 auto-enhance (CLAHE + gamma + WB + studio bg)	REAL	Classical approximation of a Zero-DCE-style result. <i>Can over-process an already-good photo – see 8.5.</i>
F2 ASR (speech -> text)	REAL if faster-whisper installed or an ASR API key set; otherwise FALLBACK to a typed transcript or a fixed demo transcript keyed by language. IndicWav2Vec/IndicTrans2 = documented upgrade.	
F2 slot extraction + Fact-Grounding Guard	REAL	Deterministic rule engine over the transcript + Craft-KG vocabulary. Core contribution; runs live.
F2 cross-modal authenticity check	PROTOTYPE	Colour/category heuristic. A CLIP-class visual-entailment model is the upgrade.
F2 copy generation	REAL templates; LLM only if <code>SIH_LLM_ENABLED</code> , and only to rephrase validated slots.	
F3 craft graph + recursive CTE + passport confidence	REAL	Seeded from realistic GI/ODOP/cluster data.

Component	Mode	Note
F3 certificates	SIMULATED_DATA	Every record is labelled “Sample Verification Record – not a real GI certificate”.
F4 cost-floor formula	REAL	Unit-tested to bind.
F4 price model	REAL LightGBM + tree-SHAP, trained at setup on a synthetic-but-realistic generator;	
	FALLBACK to a cost-plus-market formula if the booster file is missing.	
F5 forecast model	REAL LightGBM quantile regression;	
	SIMULATED_DATA history (flagged in the UI).	
F6 allocation	REAL	OR-Tools CP-SAT MILP, live, objective value + solve time shown.
F6 Shapley payment split	REAL	Exact enumeration of all 2^N sub-coalitions ($N \leq 8$). Monte-Carlo sampling is the stated path for large N .
F7 retrieval + fairness re-rank	REAL	TF-IDF cosine + the spec formula; weights configurable; one impression persisted per shown listing.
F7 interaction / exposure history	SIMULATED_DATA	Seeded with a deliberately skewed popular-vs-new distribution.
F7 Business Copilot	REAL	Cards built from structured F1/F4/F5/F6 output; LLM (if enabled) only phrases.
F8 verification state machine + reliability + risk flag	REAL	Deterministic; complaint triage attaches an advisory flag and routes to human review.
F8 B2B milestone payments + order commitment	PROTOTYPE / SIMULATED	Explicitly labelled “Prototype Payment Flow / Simulated”. No real banking or escrow.

Not implemented (no fake stand-in): trained IndicWav2Vec/IndicTrans2 checkpoints, a fine-tuned visual-entailment model, live market-price scraping, ONDC/GeM adapters, on-device offline AI, SMS OTP delivery, a real payment gateway/escrow.

5. What external APIs / keys you need

5.1 To run the prototype: NONE

The backend boots with **zero configuration**. SQLite is a local file, every AI service has a deterministic fallback, OTP is mocked (123456 for every number), and the two tiny ML models are trained locally at setup in about five seconds. No account, no API key, no internet required after **pip install**.

5.2 Optional integrations (each purely additive)

Capability	Env var(s)	Effect if set	If not set
Real speech recognition (F2)	install faster-whisper (pip) or SIH_ASR_BACKEND=api + SIH_LLM_API_KEY (OpenAI-compatible)	Your actual voice note is transcribed	Fixed demo transcript by language
LLM phrasing (F2 copy, Copilot)	SIH_LLM_ENABLED=true , SIH_LLM_API_KEY , SIH_LLM_BASE_URL , SIH_LLM_MODEL	Validated facts are rephrased more naturally	Deterministic templates
PostgreSQL + pgvector	SIH_DATABASE_URL=postgres://postgres:postgres@localhost:5432/sih (or docker compose up)	Full sync Postgres	Local sqlite
OTP value	SIH MOCK_OTP (default 123456)	Change the accepted code	123456
JWT secret	SIH_JWT_SECRET	Required for any non-demo use	dev-only default
Tuning weights	SIH_PRICING_MIN_MARGIN , SIH_F6_LAMBDA , SIH_F7_ALPHA/BETA/GAMMA/DELTA	Adjust fairness / trade-offs	Spec defaults

5.3 Mobile-side “API” it must reach

The Flutter app must be able to reach the backend. Passed at launch:

- Android emulator: `--dart-define=API_BASE_URL=http://10.0.2.2:8000/api --dart-define=MEDIA_HOST=http://10.0.2.2`
- Physical device on the same Wi-Fi: use the host machine’s LAN IP instead of 10.0.2.2, and start Uvicorn with `--host 0.0.0.0`.

10.0.2.2 is the Android emulator’s alias for the host machine’s `localhost`. If the backend is not running, every API-backed screen shows “No connection to the server” (see 8.1).

6. Backend API reference

Base prefix `/api`. Interactive docs at `http://localhost:8000/docs`.

6.1 Core (F1–F7)

Method	Path	Feature
POST	<code>/auth/otp/request</code> , <code>/auth/otp/verify</code>	mock OTP -> JWT
GET	<code>/auth/me</code> , <code>/auth/demo-accounts</code>	identity
GET	<code>/meta/features</code> , <code>/health</code> , <code>/reference</code>	feature matrix + pickers
POST	<code>/product</code> , <code>/product/publish</code>	product lifecycle
GET	<code>/products</code> , <code>/product/{id}</code>	list / detail
POST	<code>/product/analyze</code>	F1 readiness
POST	<code>/product/{id}/select-image</code> , <code>/product/{id}/inventory</code>	image + stock
POST	<code>/catalog/generate</code>	F2 cataloger
GET	<code>/craft/{id}</code> , <code>/crafts</code> , <code>/passport/{product_id}</code>	F3
POST	<code>/price/predict</code>	F4
GET	<code>/demand/forecast</code>	F5

Method	Path	Feature
GET	/requirements, /requirement/{id}; POST /requirements	B2B requirements
GET	/buyers	buyer directory
POST	/buyer/match, /order/allocate	F6 allocation + commit
GET	/orders, /order/{id}, /artisan/{id}/orders	orders
GET	/search, /search/compare; POST /search/click	F7 discovery
GET	/artisan/{id}/insights, /artisan/{id}/dashboard	F7 copilot / home
GET	/debug/ai-runs, /debug/ai-runs/summary, /debug/f7-explain	judge screen
POST	/demo/load, /demo/reset; GET /demo/scenario, /demo/trust	demo mode

6.2 Trust & Verification (F8, new)

Method	Path	Purpose
GET	/trust/artisan/{artisan_id}	public Verified Artisan card (no private KYC)
GET	/trust/business/{buyer_id}	public Verified Business card
GET	/trust/profile/{subject_id}	owner view: status + evidence + reviews
POST	/trust/verify/submit	submit an evidence item
GET	/trust/reviewer/queue	reviewer dashboard list + complaints + counts
GET	/trust/reviewer/{profile_id}	reviewer detail (evidence, F1/F2/F3 signals, complaints, timeline)
POST	/trust/reviewer/{profile_id}/actions	approve / request more evidence / reject / place under review / suspend / reinstate
GET	/trust/complaints; POST /trust/complaints	list / create a complaint (attaches an AI risk flag)
POST	/trust/complaints/{id}/resolve	resolve + optional suspend/reinstate
GET	/trust/reliability/{artisan_id}	explainable reliability breakdown
GET	/order/{order_id}/commitment	get/create the simulated commitment + milestones
POST	/order/{order_id}/commitment/advance	mark the advance milestone paid (simulated)
POST	/order/{order_id}/commitment/advance-stage	move to the next commitment stage
POST	/order/{order_id}/commitment/cancel	cancel + stage-aware consequence

All errors return a human-readable `detail` string, never a raw stack trace. Unhandled exceptions become a friendly 500: “We couldn’t process that right now. Your work is safe.”

7. User flows

7.1 Artisan – create a listing (the main F1–F7 demo)

1. Splash (animated “Karigar” wordmark) -> Onboarding -> Login. Demo artisan: phone 9800000001, OTP 123456. Or Login -> “Open SIH Demo Mode” -> Load demo.
2. **Dashboard** (“Your business today”): stat tiles, “needs your attention” Copilot cards, quick actions (Add product, Market, Inventory, Market demand, Find buyers), a verification status strip.
3. + (**centre FAB**) -> name the product, pick the craft (e.g. Madhubani painting).
4. **Product Studio (F1)**: pick a photo -> readiness score (~55–60 on the dim demo photo) -> Auto-Enhance -> Before/After + the five component bars.
5. **Add voice (F2)**: record or “Use the demo recording” -> transcript, structured attributes, the AI verification panel (origin claim flagged – “needs GI documentation”), EN/HI catalog
 - SEO keywords.
6. **Craft Passport (F3)**: provenance confidence ~0.74–0.84 with the weighted breakdown, related crafts (recursive CTE), sample verification record, GI status.
7. **Fair price (F4)**: enter material/labour/packaging/logistics -> floor, competitive range, recommended, “why this price?” + expandable tree-SHAP. Recommendation clamps to the floor if the market is below cost.
8. **Review & publish** -> listing created.
9. **Market** (dashboard quick action) -> search “handwoven cotton dupatta” -> the brand-new artisan ranks above established sellers -> “Compare” shows fairness ON vs OFF + exposure gap.
10. **Business Copilot**: “demand rising” / “you may be underpricing” cards, each tagged with the feature that produced it.

7.2 Buyer – bulk order via cluster pooling (the F6 demo)

1. Login as buyer 9900000001 / 123456, or Profile -> “Switch to Buyer mode”.
2. **Buyer home**: “Post requirement” or “Browse market”. Open the seeded requirement “5,000 bamboo welcome baskets”.
3. **Find artisan cluster** -> F8 eligibility funnel (craft -> capacity -> verification -> reliability -> eligible) -> **F6 CP-SAT MILP** runs live -> 5,000 / 5,000 across 3 artisans in ~40–90 ms.
4. **Optimization** panel: objective value, solve time, capacity utilisation. **Fair payment distribution**: Shapley bar vs by-units bar (visibly different). **vs. naive greedy baseline**.
5. **Confirm allocation** -> allocations committed; artisan sales/reliability updated.
6. **Order protection & payments** (buyer): the simulated commitment flow – stages, milestone payments 20/30/30/20, “Pay advance”, “Advance to next stage”, “Cancel order” with a stage-aware consequence. Labelled “Prototype Payment Flow / Simulated” throughout.
7. Per artisan on the allocation screen: **Report an issue** -> complaint form.

7.3 Trust & Verification (the second, trust-focused demo)

1. Profile -> “My verification” (artisan): status banner, “what we have checked” badges, your proofs, “add one more proof” (icon-led, low-literacy).
2. Profile -> Demo controls -> **Verification reviewer (platform)**: queue with counts; pick “Rupa Yadav” (seeded PENDING with one open complaint).
3. Reviewer detail: submitted evidence, per-product F1 readiness / F2 claims / F3 passport + GI status, order history, reliability breakdown, timeline. Actions: Approve / Request more evidence / Reject / Place under review / Suspend / Reinstate.
4. **Approve** -> Identity + Craft verified, GI pending -> badge becomes VERIFIED ARTISAN.
5. As a buyer, **Report an issue** (“Product not as described”) -> the backend records the complaint, attaches an **AI risk flag** (advisory), and moves the account to UNDER_REVIEW.
6. Reviewer investigates -> **Suspend** (temporary) or **Reinstate**. The final call is human. GET /api/demo/trust returns the live state + the script.

7.4 Language

Login screen (top-right) or Settings -> Language, or Profile -> Demo controls -> Language. 23 entries (English + 22 Eighth-Schedule languages). 12 fully translated (Hindi, Bengali, Tamil, Telugu, Marathi, Gujarati, Kannada, Malayalam, Punjabi, Odia, Assamese, Urdu – Urdu renders right-to-left). The other 10 are selectable and fall back to English until a native speaker contributes a dictionary. Choice persists across restarts.

8. Known issues – what errors, why, and the fix

8.1 “No connection to the server” when you tap Login / any API screen

Cause: the backend is not running, or the app was launched with the wrong `API_BASE_URL`. Pure-UI screens work; anything that calls the API errors. This is the most common “some buttons give an error” report.

Fix: start Uvicorn first (section 9), verify `http://localhost:8000/api/health` returns `{"status":"ok"}`, and launch the app with the `--dart-define` values in section 5.3. On a physical device use the LAN IP and `uvicorn --host 0.0.0.0`.

8.2 The camera shows a moving 3D room, not a real photo

Cause: the Android emulator’s default camera renders a synthetic “VirtualScene”, not your webcam. Not an app bug.

Fix: emulator toolbar -> ... (Extended controls) -> Camera -> set Front and Back to `Webcam0`. Or use “Choose from gallery” with a real craft photo (drag an image onto the emulator to add it).

8.3 Voice note always produces the same transcript

Cause: by design, in the default setup. `faster-whisper` is not installed and no ASR key is set, so F2 returns a fixed transcript keyed only by the language hint (Hindi / Maithili / Assamese). Everything downstream (grounding, EN/HI copy) then runs for real on that text.

Fix (optional): `pip install faster-whisper` in the backend venv, or set `SIH_ASR_BACKEND=api` + `SIH_LLM_API_KEY`.

8.4 “Pay advance” / “Order protection & payments” – is this real money?

No. The entire commitment + milestone-payment + cancellation flow is a UI/data simulation, labelled “Prototype Payment Flow / Simulated” on every screen. The endpoints work (verified, 0 failures) – they move state and compute consequences – but no payment provider or escrow is integrated. Production connects a real gateway here.

8.5 The AI photo enhancer can make a good photo worse

Cause: auto-enhance runs the full pipeline (CLAHE + grey-world white balance + synthetic background) whenever the score is 50–79, not just the weak dimension. On an already-good photo: CLAHE can over-sharpen and add halos; grey-world WB can shift genuine warm/coloured craft tones toward grey; the fake studio background can bite into product edges from an imperfect GrabCut mask. There is a guard (“keep only if the score does not drop”), but the score has no colour-fidelity term, so a visibly worse image can still pass.

Status: identified, not yet changed (per instruction). Safe fix available: gate CLAHE behind low measured contrast, gate/drop grey-world WB, only replace the background when `background_cleanliness` is actually low (and feather the mask), and reject the enhanced version if any single dimension drops more than a few points.

8.6 A buyer cannot place a small direct order (1–10 pieces)

Cause: the buyer side currently supports only bulk B2B via requirement + cluster pooling (F6). Product detail is view-only for buyers – no quantity picker, no “Order” button, and there is no small-order endpoint.

Status: identified, not yet built. Planned minimal, backward-compatible addition: `POST /product/{id}/order{quantity}` reusing the existing `Order` / `OrderAllocation` / `Inventory` tables (no schema change), plus a quantity stepper + “Place order” on the buyer’s product detail screen and a product list on the Verified Artisan card.

8.7 Delivery / logistics tracking is partial

Order status and a `delivery_status` field + a stage tracker UI exist; actual courier integration is intentionally mocked (as the spec allows).

8.8 Cosmetic items already fixed this iteration

- Dashboard stat cards overflowed by ~7.5 px in some languages -> fixed (grid aspect ratio + `maxLines`/ellipsis).
- Bottom navigation FAB notch cut into a label with 5 tabs -> fixed (now 4 tabs: Home / Products / [+] / Orders / Profile; “Market” moved to a dashboard quick action).
- `record` audio package pulled a Linux sub-package that failed to compile on Android -> fixed (`record` bumped to 6.x, a self-consistent set).

9. Setup and run

9.1 Prerequisites

- Python 3.12+ (3.14 works), `pip`.
- Flutter 3.27+ (built on Flutter 3.47 / Dart 3.13).
- Optional: Docker (Postgres path), an Android emulator or device.

9.2 Backend

```
cd backend
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements.txt
python -m app.ai.f4_pricing.train # ~5 s, writes data/models/*.txt
python -m app.ai.f5_demand.train
python -m app.db.seed           # builds the demo SQLite DB
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The server auto-seeds on first boot if the DB is empty. Health: `http://localhost:8000/api/health`.

9.3 Mobile

```
cd mobile
flutter pub get
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8000/api ^
            --dart-define=MEDIA_HOST=http://10.0.2.2:8000
```

Login: artisan 9800000001 / 123456, buyer 9900000001 / 123456. In-app: Login -> “Open SIH Demo Mode”
-> Load demo.

9.4 Windows-specific: Android build fails with “different roots”

If the project is on E: and the Dart/Flutter pub cache is on C:, Kotlin incremental compilation throws `java.lang.IllegalArgumentException: this and base files have different roots` (Windows treats each drive letter as a separate filesystem root, so relative-path math across drives fails). This build already includes the fix – these two lines in `mobile/android/gradle.properties`:

```
kotlin.incremental=false
kotlin.incremental.useClasspathSnapshot=false
```

If it still fails, align the drives: set a user environment variable `PUB_CACHE=E:\PubCache`, reopen all terminals, `flutter pub get`, then `flutter clean` and run again – or copy the project to the same drive as the pub cache.

9.5 Tests

```
cd backend && python -m pytest -q          # 33 tests
cd backend && python scripts/smoke.py      # 43-check end-to-end
cd mobile && flutter analyze                # 0 issues
cd mobile && flutter test                  # model-parsing tests
cd mobile && flutter build web             # full-tree compile check
```

10. What changed in this iteration

10.1 New – Trust & Verification (F8)

- **Backend models** (backend/app/models/trust.py, all new tables, zero ALTER): `VerificationProfile`, `VerificationEvidence`, `VerificationReview`, `Complaint`, `TrustEvent`, `B2BOrderCommitment`.
- **Service** (backend/app/services/trust.py): profile bootstrap, public card projections (no private KYC), explainable reliability score, trust-status state machine with audit trail, AI risk-flag path (flag -> `UNDER_REVIEW` -> human).
- **Routes** (backend/app/api/routes/trust.py): public cards, owner profile, evidence submit, reviewer queue/detail/action, complaints CRUD + resolve, reliability, B2B commitment get/advance/advance-stage/cancel.
- **F6 integration** (backend/app/services/helpers.py, backend/app/ai/f6_matching/allocation.py, backend/app/api/routes/orders.py): verification-status eligibility gate before the MILP, eligibility funnel trace, per-allocation verification + reliability in the response.
- **Seed** (backend/app/db/seed.py): 20 verification profiles (16 artisans + 4 businesses, all marked Demo Data), evidence rows, one open demo complaint against “Rupa Yadav”.
- **Demo** (backend/app/api/routes/demo.py): `GET /api/demo/trust` + a `trust_scenario` block on `/api/demo/scenario`. F8 added to `/api/meta/features`.
- **Tests** (backend/tests/test_trust.py): 6 new tests (cards, state machine + risk flag, reliability explainability, F6 gate, simulated commitment).
- **Mobile** (mobile/lib/features/trust/): models, API methods, `VerifiedCardView` + `TrustBadgeChip` widgets, screens for `/verification`, `/reviewer`, `/reviewer/:id`, `/trust/artisan/:id`, `/trust/business/:id`, `/report/:artisanId`, `/order-commitment/:orderId`; verified badges surfaced on profile, dashboard, allocation cards, product detail; buyer app-bar link.

10.2 New – 22-language localization

- `mobile/lib/l10n/i18n.dart` (`LocaleProvider` + `tr()` + 23 locales), `mobile/lib/l10n/langs.dart` (12 hand-written dictionaries), `mobile/lib/features/settings/language_screen.dart` (picker).
- `pubspec.yaml`: added `flutter_localizations`, intl bumped to `^0.20.2`.
- `main.dart` wired for locale + `MaterialApp` rebuild on language change.
- `tr(...)` applied across nav, dashboard, login, settings, profile, products, orders, buyer flow, allocation, passport, pricing, demand, search and the trust screens.

10.3 UX + build fixes this iteration

- Bottom navigation -> 4 tabs + centre FAB (`app_shell.dart`, `app_router.dart`).
- Splash -> animated typing of the “Karigar” wordmark; splash controls its own exit (`splash_screen.dart`, redirect tweak in `app_router.dart`).
- Dashboard stat-card overflow fixed; “Market” quick action added (`dashboard_screen.dart`).
- `catalog_preview_screen.dart`: post-publish navigation adjusted for the new route layout.
- `record` -> `^6.0.0` to fix an Android Kotlin compile error from a broken transitive Linux sub-package.
- `mobile/android/gradle.properties`: Kotlin incremental compilation disabled to fix the cross-drive build failure on Windows.

11. Production upgrade path (brief)

Feature	Prototype now	Production
F1	GrabCut + classical scoring	MobileSAM/SAM2 on-device, NIMA-class IQA, quantised Zero-DCE
F2	pluggable ASR + rules	IndicWav2Vec + IndicTrans2, fine-tuned NER, CLIP
F3	recursive CTE	visual-entailment, moderation queue Neo4j only if queries outgrow CTEs; learned link prediction for GI candidacy
F4	synthetic-trained LightGBM	LightGBM on real cost/sale data + live market-price scraping
F5	simulated history	retrain on 3–6 months of real transactions; MASE monitoring
F6	exact Shapley ($N \leq 8$)	Monte-Carlo Shapley; real-time re-solve on decline at SLA
F7	TF-IDF + static weights	click-log-driven learned relevance; doubly-stochastic fairness LP
F8	evidence + state machine + simulated payments	government KYC / GI register integration; real payment provider + escrow; courier APIs
Infra	monolith + SQLite	split services, Postgres+pgvector, ONDC seller-app + GeM catalogue adapters
i18n	12 hand dictionaries + fallback	complete all 22 with native reviewers; localise F2 output beyond EN/HI